TASK 1 – Understand the Problem and Identify the Data

Problem analysis:

Grama Jyothi Rural Electricity Co-operative bills households by comparing the previous and current meter readings. Consumption is the number of units used during the billing period. The bill contains a fixed charge and an energy charge calculated using slabs: the first 100 units are free, units 101–200 cost ₹2.25 per unit, and units above 200 cost ₹4.50 per unit. The fixed charge is ₹50 in the supplied code example.

The program must avoid negative bills, calculate money exactly to the paisa, handle a 16-bit meter rollover, and keep tariff values in one place so a tariff revision cannot be applied inconsistently

Data item	Purpose and justification	Suggested representation
Household number	Identifies the customer and the bill; must not be confused with consumption.	int or a suitably sized integer; string if it contains letters/leading zeroes
Category	Stores household/customer tariff category if different tariffs are later introduced.	enum or small integer
Previous reading	Meter register value at the start of the period; in the stated example it is 0–65535.	unsigned short / uint16_t where supported

Current reading	Meter register value at the end of the period; same register range.	unsigned short / uint16_t
Units consumed	Difference between readings, including rollover logic.	unsigned int or uint32_t
Energy charge	Sum of the applicable slab charges.	long long integer paise
Fixed charge	Fixed amount added to each bill; constant under a given tariff.	long or long long integer paise
Total bill	Fixed charge plus energy charge; must be non-negative and exact to the paisa.	long long integer paise
Tariff rates and slab limits	Shared rules that should not be repeated throughout the program.	named macros or const values; configuration file in revised design
Number of bills / total collection	May grow from 4,000 to 800,000 households.	size_t for counts; long long for total paise

Inputs, outputs, rules and assumptions:

- Inputs: household number, category, previous reading and current reading. Input validation must reject missing, malformed or out-of-range values.
- Outputs: units consumed, energy charge, fixed charge and final bill in rupees and paise.
- Rules: total bill = fixed charge + energy charge; all charges are non-negative; the first 100 units have zero energy charge; slab boundaries are applied exactly once; a rollover is handled according to the meter register's modulus.

- Assumption: the basic example uses a 16-bit register with values 0–65535 and one rollover between readings. Multiple rollovers cannot be inferred from only two register readings.
- Assumption: the sample tariff and fixed charge are those shown in the assignment code. Real-world tax, subsidies, late fees and category-specific rules are outside the supplied brief.

Text strings versus integers: time and space:

If a reading is stored as text such as "01530", the program must validate the characters and convert the digit sequence to a numeric value before calculating consumption. Conversion and comparison each take O(d) time for d digits. A fixed-width integer comparison/subtraction is O(1).

Representation	Time per reading	Space per reading	Comments
Integer meter value	O(1) arithmetic	Fixed size, commonly 2 bytes for unsigned short	Best for calculations; leading zeroes are not retained.
Text such as "01530"	O(d) to validate/convert; O(d) to compare as text if numeric meaning is needed	At least d characters plus terminator for a C string	Preserves formatting, but requires conversion and validation.

For the assignment's small fixed register, integer storage is simpler and faster. Text input may still be read as a line and then safely converted, but the value used for arithmetic should be numeric.